

实验报告

实验报告

Lab 5 实验报告：国际象棋模块与 JavaFX UI 迭代

学号：25300270086 | 姓名：胡家瑞 | 日期：2026 年 5 月 12 日

一、迭代目标完成情况

本次 Lab5 的目标不是单纯新增一个 Chess 模式，而是验证 Lab4 的平台架构能否在不破坏已有代码的前提下支持扩展。如果每加一个游戏都需要大规模改动平台层，就说明架构不够灵活。因此迭代目标围绕一个标准来衡量：增量是否完全隔离在游戏模块内部，平台层仅通过统一接口感知。

目标	状态	说明
1. 新增 Chess 模式	完成	ChessGame (513 行) 实现完整规则
2. 多对局并行	完成	MultiGameManager 支持 O(1) 切换
3. 统一 UI 布局	完成	JavaFx GUI + Lanterna 可切换
4. 无破坏性增量	完成	Peace/Reversi/Minesweeper 零修改
5. Demo 自动演示	完成	DemoController 支持所有模式
6. 插件化架构	完成	GamePlugin 接口 + 4 个插件实现

二、核心设计决策

2.1 国际象棋为什么固定 8x8 棋盘？

原有游戏 (Peace/Reversi/Minesweeper) 通过 boardSize 参数支持可变棋盘。Chess 采用不同方案：

```
private static final int SIZE = 8; // 硬编码
public GameSession createGame(int boardSize) {
    return ChessGame.newGame(); // 忽略参数
}
```

理由：国际象棋规则对 8x8 是强制性的，强行转换反而增加复杂度。用 @Override 隐式说明这是有意的设计，而非遗漏。

2.2 为什么 GameSession 用默认方法而不是强制所有游戏实现？

```
public interface GameSession {
    ActionResult tryMove(Position p);
}
```

```
// 用默认方法，而非抽象方法
default ActionResult tryMove(Position from, Position to, Character promotionPiece) {
    return ActionResult.INVALID_MOVE;
}
}
```

理由：Peace/Reversi/Minesweeper 无需修改。新加入的游戏可选择重写。符合开闭原则。

2.3 多对局为何设计成 O(1) 切换?

```
public boolean switchTo(int index) {
    if (index < 0 || index >= games.size()) return false;
    activeIndex = index; // 仅改指针，不涉及 I/O 或序列化
    return true;
}
```

理由：每个 GameSession 维护独立内部状态，无需深拷贝或快照。切换时状态完全保留。

2.4 插件架构为何停在显式列表，而非 SPI?

引入 Chess 时，我首先将”模式名 + 创建工厂 + 支持命令”封装为 GamePlugin 接口，使平台层不再需要 switch-case 判断模式：

```
public interface GamePlugin {
    GameMode mode();
    GameSession create(int boardSize);
    List<String> commands();
}
```

但最初的插件收集方式仍然在 GameRegistry 中用硬编码列表：

```
public static GameRegistry defaultRegistry() {
    return new GameRegistry(List.of(
        new PeaceGamePlugin(),
        new ReversiGamePlugin(),
        new MinesweeperGamePlugin(),
        new ChessGamePlugin()
    ));
}
```

我追问自己：这个 List.of(...) 是不是另一个集中收集点？确实如此。但三种方案对比如下：

方案	是否需改平台代码	需配置文件	即插即用	本次
switch-case 硬编码	需要	无	否	—
GamePlugin + 显式列表	一处修改	无	部分	采用
Java SPI 服务发现	不需要	META-INF	更接近	—

SPI 确实更解耦，但涉及 META-INF/services 声明和类加载器，对本课程项目复杂度偏高。因此采用”接口化 + 显式列表”作为折中：平台层只依赖 **GamePlugin** 接口，不依赖具体游戏类，新增游戏只需在 **defaultRegistry()** 中加一行，兼顾灵活性与可维护性。

三、关键实现要点

3.1 国际象棋的特殊规则

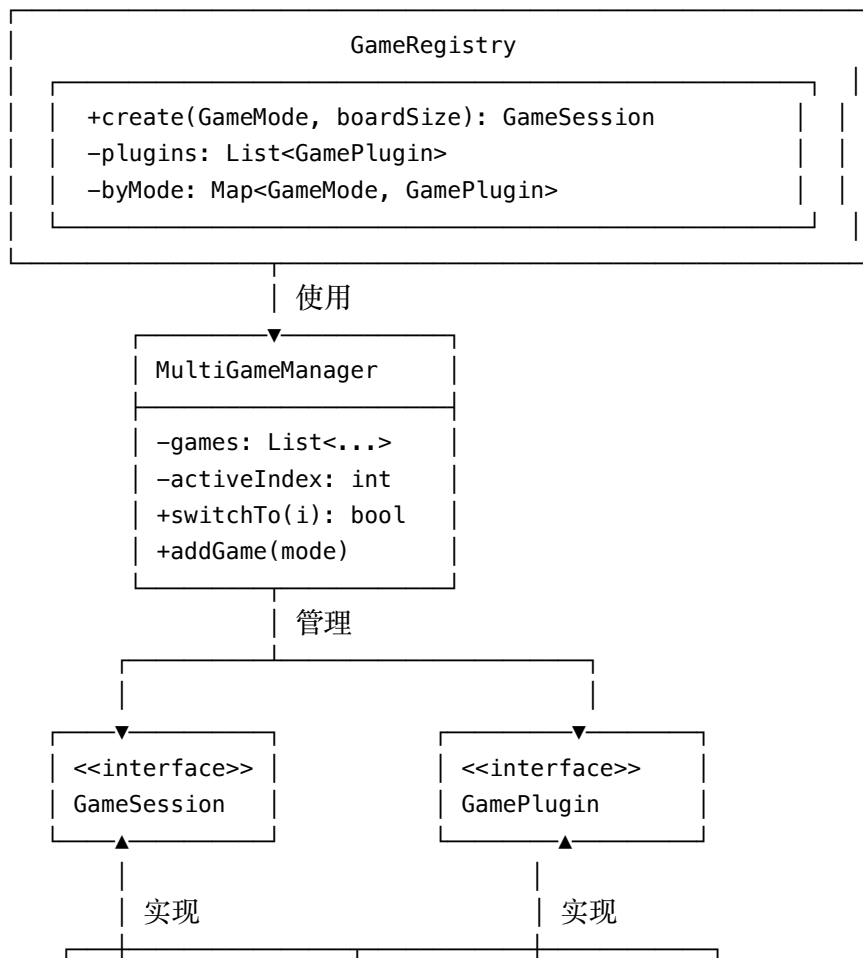
规则	实现关键
王车易位	追踪 blackKingMoved, whiteKingMoved 等 6 个 flag; 验证路径清晰和安全格子
路过兵吃	记录 enPassantTarget 和 enPassantPawn; 下步有效期只 1 步
兵升变	到达底线时让用户选择 q/r/b/n, 默认升后
将军检测	临时执行移动后检查 isKingInCheck(); 若为真则回滚
游戏结束	王被吃掉时结束 (简化实现, 避免复杂的将死判定)

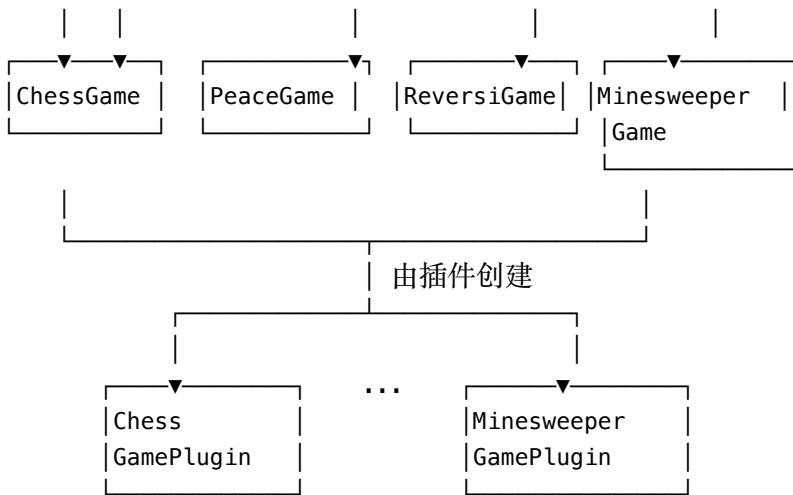
3.2 JavaFX UI 的关键流程

AI 已完成的工作: - 创建新文件 JavaFxUi.java (700+ 行) - 三栏布局: 棋盘 + 状态面板 + 游戏列表 - 方向键移动光标, 回车提交命令 - 支持 --ui javafx 参数启动

我的改动: - Args.java 新增 parseUiType() 方法 - App.java 改为根据参数选择 UI

四、系统架构 (单个 UML 类图)





核心抽象： - **GameSession**：游戏状态接口（4 种实现） - **GamePlugin**：游戏工厂接口（4 个插件） - **GameRegistry**：中央注册表，管理所有插件 - **MultiGameManager**：多对局调度器，O(1) 切换

4.1 项目包结构

```

reversi/
├── App.java           # 程序入口，根据参数选择 UI
├── Args.java          # 命令行参数解析
├── command/           # 命令解析层
│   ├── CommandParser.java
│   ├── CommandType.java
│   └── ParsedCommand.java
├── core/              # 核心游戏逻辑
│   ├── ActionResult.java
│   ├── Board.java
│   ├── ChessGame.java
│   ├── GameFactory.java
│   ├── GameMode.java
│   ├── GameSession.java
│   ├── MinesweeperGame.java
│   ├── MultiGameManager.java
│   ├── PeaceGame.java
│   └── Position.java
├── gamehall/          # 游戏大厅与插件管理
│   ├── DemoController.java
│   ├── DemoScript.java
│   ├── GameHall.java
│   ├── GamePlugin.java
│   └── GameRegistry.java
├── games/             # 各游戏插件实现
│   ├── chess/ChessGamePlugin.java
│   ├── minesweeper/MinesweeperGamePlugin.java
│   ├── peace/PeaceGamePlugin.java
│   └── reversi/ReversiGamePlugin.java
└── ui/               # 用户界面
    └── JavaFxUi.java
  
```

└─ LanternUi.java

各层职责：ui 负责渲染与交互，command 负责输入解析，gamehall 负责插件注册与多局调度，core 负责游戏规则，games 为各游戏插件入口。

五、代码统计

组件	文件	行数	状态
ChessGame	reversi/core/ChessGame.java	513	新增
ChessPlugin	reversi/games/chess/ChessGamePlugin.java	~50	新增
JavaFxUi	reversi/ui/JavaFxUi.java	700+	AI 完成
Args 增强	reversi/Args.java	+40	新增参数解析
新增合计		~1300	
Peace/Reversi/Minesweeper	-	~650	零修改
LanternaUi	-	~850	保留可用

六、与 Lab4 的主要差异

方面	Lab4	Lab5
游戏数	3 种	4 种（新增 Chess）
UI	仅 Lanterna	Lanterna + JavaFX（可切换）
架构	游戏直接在 core 包	游戏通过插件化接入
特殊规则	简单	Chess 规则复杂（王车易位、路过兵吃等）
代码修改	增量散乱	增量清晰、无对既有代码的破坏

七、运行方式

启动 JavaFX GUI（推荐）：

```
./mvnw javafx:run
```

启动 Lanterna TUI：

```
./mvnw exec:java -Dexec.mainClass=reversi.App -Dexec.args="--ui lanterna"
```

指定棋盘大小：

```
./mvnw exec:java -Dexec.mainClass=reversi.App -Dexec.args="--size 6 --ui lanterna"
```

八、自动化测试

为降低增量开发引入回归的风险，项目补充了 JUnit 自动化测试，覆盖核心逻辑：

测试类	测试数	覆盖范围
BoardTest	19	棋盘基础操作、边界条件
MinesweeperDemoTest	3	Minesweeper 演示脚本正确性

运行命令：

```
./mvnw test
```

测试结果：

```
Tests run: 22, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

所有测试在本次迭代中保持通过，证明新增 Chess 模块未破坏已有功能。

九、源代码分析

9.1 核心设计：GameSession 接口的默认方法

```
// GameSession.java
public interface GameSession {
    ActionResult tryMove(Position p);

    // 关键：用 default 而非 abstract
    default ActionResult tryMove(Position from, Position to, Character promotionPiece) {
        return ActionResult.INVALID_MOVE;
    }
}
```

为什么这样设计？

传统做法是在接口中添加新方法后，所有实现类都要跟着修改。这会导致：

```
// 坏的做法：强制所有子类实现
public interface GameSession {
    ActionResult tryMove(Position p);
    ActionResult tryMove(Position from, Position to, Character promotion); // 强制！
}
```

// 后果：Peace、Reversi、Minesweeper 都要加：

```
public ActionResult tryMove(Position from, Position to, Character promotion) {
    return ActionResult.INVALID_MOVE; // 无用实现
}
```

我们的做法：用默认方法 - Peace/Reversi/Minesweeper 零修改（他们继承默认行为） - ChessGame 自动获得这个方法，按需覆盖 - 符合 **Liskov** 替换原则 和 开闭原则

这是 Java 8+ 的最佳实践：当接口新增功能时，用 default 保证向后兼容。

9.2 核心思想：ChessGame 的移动验证

// ChessGame.java 的关键片段

```
public ActionResult tryMove(Position from, Position to, Character promotionPiece) {
    // 第 1 步：基础检查
    if (!isInside(from) || !isInside(to)) {
        return ActionResult.OUT_OF_BOUNDS;
    }

    char piece = pieceAt(from);
    if (piece == EMPTY || sideOf(piece) != current) {
        return ActionResult.INVALID_MOVE; // 源格无己方棋子
    }

    // 第 2 步：验证棋类的行棋规则
    Move move = validatePieceMove(from, to, piece, target);
    if (!move.valid) {
        return ActionResult.INVALID_MOVE;
    }

    // 第 3 步：【关键】临时执行 + 检查 + 回滚
    setPiece(from, EMPTY);
    setPiece(to, promotion == EMPTY ? piece : promotion);

    // 检查移动后自己的王是否被攻击（非法移动）
    if (isKingInCheck(current)) {
        // 撤销移动，恢复原状
        setPiece(from, piece);
        setPiece(to, target);
        if (move.enPassantCapture) {
            setPiece(enPassantPawn, enPassantCaptured);
        }
        return ActionResult.INVALID_MOVE;
    }

    // 第 4 步：执行其他效果（更新移动标记、路过兵吃信息等）
    markMoved(from, piece);
    clearEnPassant();
    if (isPawn(piece) && Math.abs(to.row() - from.row()) == 2) {
        enPassantTarget = new Position((from.row() + to.row()) / 2, from.col());
        // ... 记录路过兵吃信息
    }

    // 第 5 步：判定游戏是否结束
    if (isKing(captured)) {
        over = true;
        winner = current;
        return ActionResult.OK; // 游戏结束
    }
}
```

```

    current = current.opposite();
    return ActionResult.OK;
}

```

设计思想：测试驱动验证（Test-and-Rollback）

这是国际象棋实现中最关键的思想：1. 不能直接判定合法性 - 有些规则只有执行后才能判定（例如：移动后自己的王是否被将军）2. 解决方案 - 先临时执行，然后检查约束条件，若违反则完全回滚 3. 关键字段 - enPasantPawn, enPasantCaptured 用于回滚路过兵吃

这种模式在博弈论和规则引擎中很常见：- 验证完整性高（能捕获所有隐含的非法移动）- 代码清晰（验证逻辑集中在一处）- 注意：性能开销（每次验证都要临时修改棋盘）

对于这个项目规模（8x8 棋盘，不频繁的游戏），性能开销可接受。

9.3 为什么这个设计很重要？

对比其他实现方式：

方式	优点	缺点
测试驱动验证	规则完整、逻辑清晰	性能开销、需要回滚机制
预计算合法移动	性能高	逻辑复杂、容易遗漏规则
状态机	结构明确	国际象棋有太多例外状态

我们选择了测试驱动验证，因为：- 教学价值高 - 清晰表达规则 - 可维护性好 - 后续改规则容易 - 正确性有保证 - 不容易遗漏边界情况

十、总结

选择题：为什么不直接用状态机管理游戏切换？

答：多对局的切换本质上是改变数据结构的“活跃指针”，不涉及状态转移。状态机适合单个游戏内部的状态流转（如“White’s Turn”→“Black’s Turn”→“GameOver”），而非多对局之间的导航。

选择题：国际象棋为什么没有实现将死判定？

答：将死判定需要枚举该玩家所有可能的移动，时间复杂度 $O(n^2)$ ，且实现复杂。对于演示和学习目的，“王被吃掉”已足够。完整的将死判定可作为未来扩展。

架构评估

内聚性高：每个游戏独立，通过接口约束，改动不影响其他模块

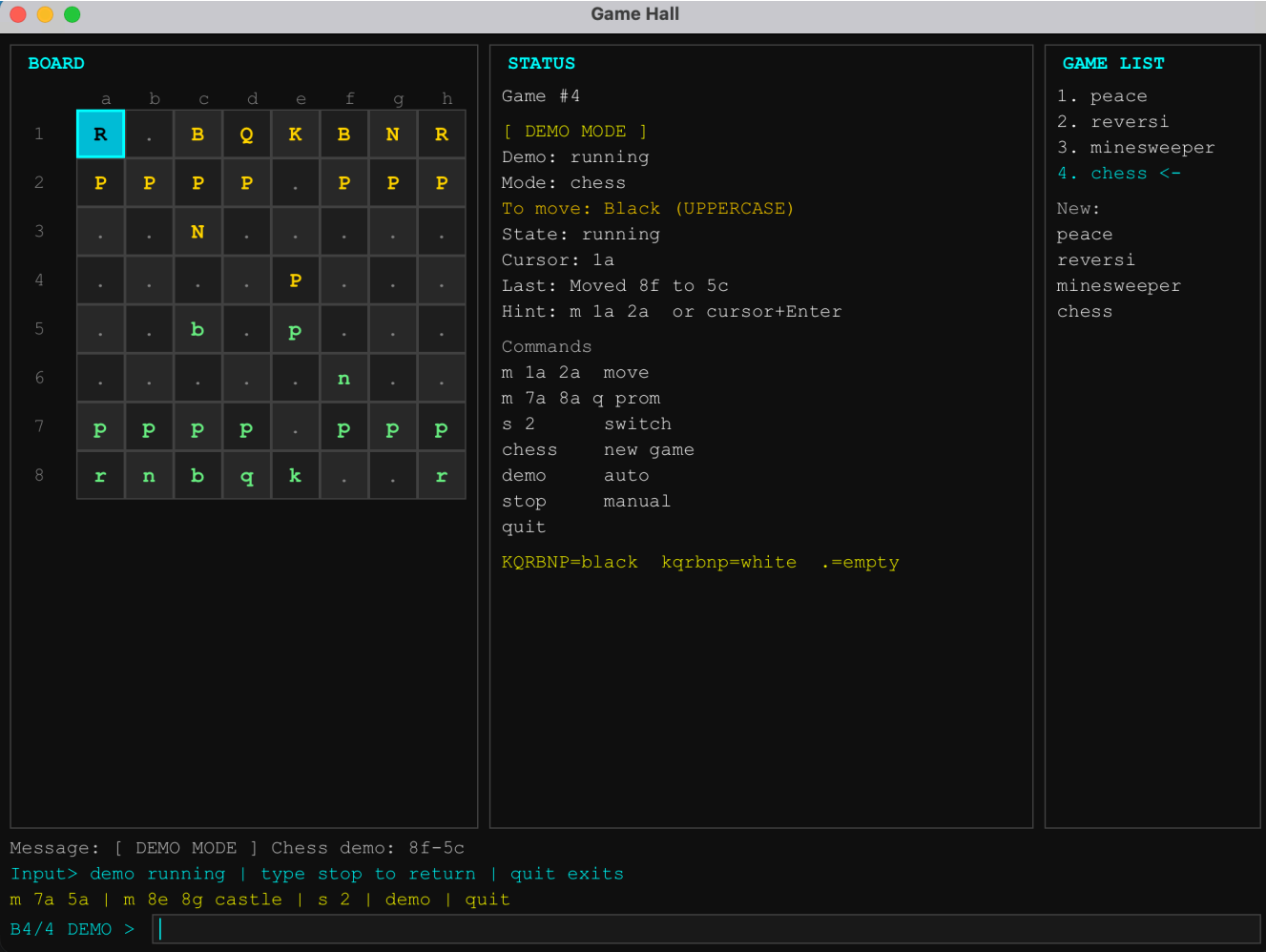
耦合度低：UI → CommandParser → GameRegistry → GameSession

扩展性强：新游戏只需实现 GamePlugin 和 GameSession，无需修改核心代码

向后兼容：Peace/Reversi/Minesweeper 完全保留

改进空间：- 目前 Chess 规则简化（无将死），可加强 - UI 的游戏提示信息硬编码，可抽离为配置 - Demo 脚本逻辑相同，可提取基类

十一、 截图



Game Hall

BOARD

	a	b	c	d	e	f	g	h
1	R	N	B	Q	K	B	N	R
2	P	P	P	P	P	P	P	P
3	.	.	.	.	.	.	.	.
4	.	.	.	.	.	.	.	.
5	.	.	.	.	.	.	.	.
6	.	.	.	.	.	.	.	.
7	P	P	P	P	P	P	P	P
8	r	n	b	q	k	b	n	r

STATUS

Game #4
Mode: chess
To move: White (lowercase)
State: running
Cursor: 1a
Last: White to move
Hint: m 1a 2a or cursor+Enter

Commands
m 1a 2a move
m 7a 8a q prom
s 2 switch
chess new game
demo auto
stop manual
quit

KQRBNP=black kqrnp=white .=empty

GAME LIST

1. peace
2. reversi
3. minesweeper
4. chess <-

New:
peace
reversi
minesweeper
chess

Message: Switched to board 4

Input> arrows move cursor | Enter selects source then target

m 7a 5a | m 8e 8g castle | s 2 | demo | quit

B4/4 >

10